BiteFixes Enterprise Architecture Manual

Volume 16

Observability & Monitoring Architecture (OMA)

Version: 1.0

Classification: Enterprise Architecture

Status: Draft

Table of Contents

16.12 Alerting Architecture

16.13 Dashboards

16.14 Capacity Monitoring

16.15 Incident Detection

16.16 Future Evolution

16.17 Volume Summary

16.1 Purpose

The Observability & Monitoring Architecture defines how the BiteFixes Platform measures, monitors, analyzes, and visualizes the operational health of every component.

The objectives are to:

- Detect problems before users are affected.
- Improve operational reliability.
- Reduce incident resolution time.
- Support AI monitoring.
- Enable data-driven operational decisions.
- Provide complete visibility across the platform.

Observability applies to infrastructure, applications, APIs, AI services, databases, integrations, and business processes.

16.2 Observability Vision

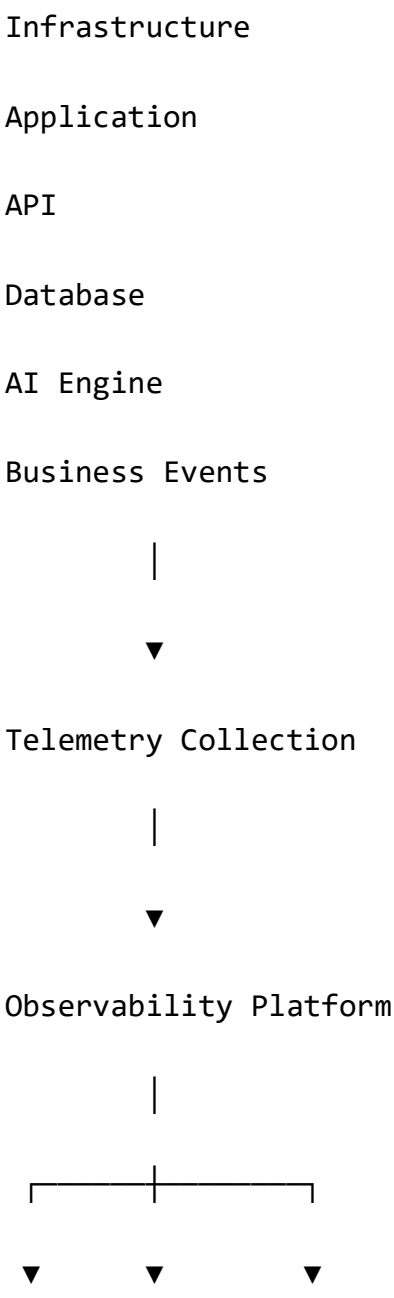
Traditional monitoring answers:

Is the system running?

Observability answers:

Why is the system behaving this way?

The platform is designed so that every component continuously produces telemetry.



```
Alerts

Analytics
```

16.3 Observability Principles

Principle 1 — Everything Produces Telemetry

Every service generates:

- Logs
- Metrics
- Events
- Traces

No critical component operates without observability.

Principle 2 — End-to-End Visibility

A single customer request can be traced across:

- API Gateway
- Authentication
- Bitey
- Business Services
- Database
- External Integrations

Principle 3 — Business and Technical Metrics

The platform monitors both technical performance and business outcomes.

Examples:

Technical:

- CPU
- Memory
- Database latency

Business:

- Tickets created
- Repairs completed
- AI accuracy
- Customer satisfaction

Principle 4 — Automation

Monitoring automatically detects anomalies and generates alerts.

Manual monitoring is minimized.

Principle 5 — Continuous Improvement

Telemetry is used to identify:

- Performance bottlenecks
- Reliability issues
- Security anomalies
- AI optimization opportunities
- Capacity planning needs

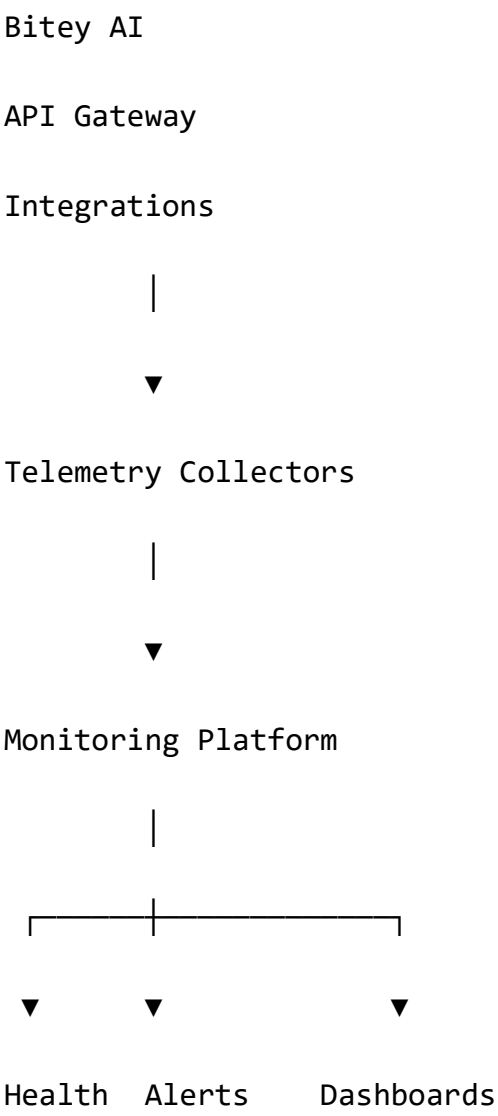
16.4 Monitoring Architecture

The monitoring platform consolidates telemetry from all platform components.

```
Infrastructure

FastAPI

Supabase
```



Monitoring Scope

The architecture covers:

- Platform availability
- API health
- Database health
- AI services
- Authentication
- Background jobs
- Integrations
- Network connectivity
- Storage
- Queue processing

Health Checks

Every critical component exposes a health endpoint.

Health states:

```
Healthy

↓

Degraded

↓

Unhealthy

↓

Unavailable
```

Health information is consumed by monitoring tools and orchestration systems.

16.5 Logging Architecture

Overview

Logs provide chronological records of platform activity.

They support:

- Troubleshooting
- Security investigations
- Compliance
- Performance analysis
- AI diagnostics

Log Categories

Application Logs

Business processing events.

Security Logs

Authentication.

Authorization.

Access attempts.

Permission changes.

Infrastructure Logs

Operating system.

Containers.

Networking.

Storage.

AI Logs

Intent detection.

Knowledge retrieval.

Confidence scores.

Prompt execution.

Memory updates.

Integration Logs

WhatsApp.

WordPress.

Email.

Payment providers.

External APIs.

Structured Logging

Every log entry contains:

- Timestamp
- Level
- Service
- Correlation ID
- Company ID
- User ID (when applicable)
- Event Type
- Message

Structured logs enable automated analysis and efficient searching.

Log Levels

`TRACE`

`DEBUG`

`INFO`

`WARNING`

`ERROR`

```
CRITICAL
```

Each level serves a specific operational purpose.

16.6 Metrics Architecture

Overview

Metrics provide quantitative measurements describing platform behavior over time.

Unlike logs, metrics are optimized for aggregation, trend analysis, and alerting.

Metric Categories

Infrastructure Metrics

Examples:

- CPU utilization
- Memory usage
- Disk capacity
- Network throughput

Application Metrics

Examples:

- Request count
- Response time
- Error rate
- Active sessions

Database Metrics

Examples:

- Active connections
- Query latency
- Slow queries
- Storage utilization

API Metrics

Examples:

- Requests per second
- Success rate
- Error rate
- Authentication failures

AI Metrics

Examples:

- Intent accuracy
- Response latency
- Knowledge retrieval success
- Confidence distribution
- Memory utilization

Business Metrics

Examples:

- Tickets created
- Repairs completed
- Active customers
- Technician productivity
- Revenue trends

Metrics Collection

```
Platform Components

↓

Metrics Exporters

↓

Metrics Collector

↓

Time-Series Database

↓

Dashboards

↓

Alerts
```

Metrics are retained according to configurable retention policies to support both operational monitoring and historical trend analysis.

Chapter Summary

The first section of the Observability & Monitoring Architecture establishes the strategic vision and foundational mechanisms for collecting telemetry across the BiteFixes platform. By standardizing monitoring, logging, health checks, and metrics, the architecture provides comprehensive visibility into infrastructure, applications, APIs, databases, AI services, and business operations.

BiteFixes Enterprise Architecture Manual

Volume 16

Observability & Monitoring Architecture (OMA)

16.7 Distributed Tracing

Overview

Distributed Tracing provides end-to-end visibility into every request processed by the BiteFixes platform. Unlike traditional logging, tracing reconstructs the complete execution path of a request as it traverses APIs, services, databases, AI components, and external integrations.

The primary objective is to answer:

- Where did the request originate?
- Which services processed it?
- How much time was spent in each component?
- Which dependency caused latency?
- Where did an error occur?

End-to-End Request Flow

```
Customer
   |
   ▼
API Gateway
```

Every step belongs to the same distributed trace.

Trace Components

Each trace consists of:

- Trace ID
- Parent Span
- Child Spans
- Service Name
- Start Time
- End Time
- Duration
- Status
- Error Information
- Correlation ID

Every service contributes spans to the same request lifecycle.

Correlation Strategy

Every incoming request receives a globally unique Correlation ID.

Example:

```
X-Correlation-ID:
8de25a72-6e3d-48d7-bf81-54b2789d8f22
```

This identifier propagates across:

- APIs
- Internal Services
- Database Operations
- AI Processing
- External Providers
- Background Workers

Trace Visualization

```
Request
│
├── Authentication (8 ms)
├── Authorization (2 ms)
├── Bitey Intent Detection (25 ms)
├── Knowledge Search (18 ms)
├── Ticket Creation (31 ms)
├── Database Commit (11 ms)
└── WhatsApp Notification (122 ms)
```

The complete execution timeline enables rapid bottleneck identification.

Trace Retention

Trace retention is configurable according to operational requirements.

Typical retention categories:

- Live traces
- Short-term diagnostics
- Long-term analytics
- Incident investigations

Distributed Trace Benefits

Distributed tracing enables:

- Root cause analysis

- Performance optimization
- Dependency mapping
- AI latency analysis
- API optimization
- Service reliability improvements

16.8 AI Observability

Overview

Because Bitey is the cognitive engine of the platform, traditional application monitoring is insufficient.

AI Observability monitors not only infrastructure but also the quality and behavior of AI decisions.

AI Processing Pipeline

```
Customer Message

↓

Language Detection

↓

Intent Detection

↓

Knowledge Retrieval

↓

Memory Search

↓
```

```
Business Rules

↓

Response Generation

↓

Conversation Update
```

Each stage emits telemetry.

AI Metrics

The platform continuously measures:

Performance Metrics

- AI Response Time
- Knowledge Lookup Time
- Memory Lookup Time
- Prompt Processing Time

Quality Metrics

- Intent Accuracy
- Confidence Score
- Knowledge Match Rate
- Hallucination Detection
- Recommendation Accuracy

Operational Metrics

- Requests per Minute
- Active Conversations
- AI Availability

- Cache Utilization
- Token Consumption

Prompt Monitoring

Each prompt execution records:

- Prompt Version
- Processing Duration
- Model Used
- Confidence Score
- Memory Sources
- Knowledge Sources
- Final Decision

This enables prompt optimization without affecting production behavior.

Knowledge Effectiveness

Knowledge Base quality is monitored using:

- Search Success Rate
- Article Usage
- Retrieval Accuracy
- Missing Knowledge Detection
- Duplicate Articles
- Obsolete Content

AI Memory Monitoring

Memory operations generate metrics including:

```
Memory Read

↓

Memory Update
```

```
↓

Memory Consolidation

↓

Semantic Search

↓

Context Injection
```

These metrics ensure that conversational memory remains effective and scalable.

AI Explainability

Future enterprise editions will include explainability mechanisms allowing administrators to understand:

- Why an intent was selected
- Which knowledge article was used
- Which memories influenced the decision
- Confidence calculation factors

This capability improves trust and facilitates auditing.

16.9 Infrastructure Monitoring

Overview

Infrastructure Monitoring ensures that the physical and cloud resources supporting the platform remain healthy and performant.

Monitored Components

```
Servers

Virtual Machines

Containers

Storage

Networks

Load Balancers

Databases

Cache

Queues
```

Resource Metrics

Infrastructure metrics include:

CPU

- Utilization
- Load Average
- Context Switches

Memory

- Utilization
- Available Memory
- Swap Usage

Storage

- Capacity
- IOPS
- Read Latency
- Write Latency

Network

- Bandwidth
- Latency
- Packet Loss
- Connection Count

Service Availability

Critical services continuously report:

- Uptime
- Health Status
- Startup Time
- Failure Count
- Restart Count

Container Monitoring

Future containerized deployments monitor:

- Running Containers
- Restart Events
- Resource Limits
- Image Versions
- Node Allocation

16.10 Application Performance Monitoring (APM)

Overview

Application Performance Monitoring provides deep visibility into software execution.

Unlike infrastructure monitoring, APM focuses on application behavior.

Monitored Areas

```
API

↓

Business Logic

↓

AI Engine

↓

Database

↓

External Services

↓

Response
```

Key Metrics

- Response Time

- Throughput
- Error Rate
- Exception Count
- Active Requests
- Slow Transactions

Database Performance

Measured indicators include:

- Query Duration
- Connection Pool Usage
- Deadlocks
- Transaction Duration
- Lock Wait Time

Dependency Monitoring

External dependencies are continuously evaluated.

Examples:

- WhatsApp Cloud API
- Email Services
- Payment Providers
- WordPress
- Push Notification Services

Dependency latency is tracked separately from internal processing.

Performance Baselines

Historical baselines enable anomaly detection.

Examples:

- Average Response Time

- Daily Peak Requests
- AI Processing Time
- Database Latency

Significant deviations automatically trigger investigations.

16.11 Business Monitoring

Overview

Business Monitoring complements technical monitoring by observing operational indicators directly related to business performance.

This layer allows managers to understand not only whether the system is functioning, but whether the business itself is operating efficiently.

Operational KPIs

Examples include:

- Tickets Created per Day
- Tickets Closed per Day
- Average Resolution Time
- Technician Productivity
- Customer Satisfaction
- AI Automation Rate
- Knowledge Base Utilization
- Customer Retention
- Revenue by Service
- Average Repair Cost

Executive Dashboard Flow

```
Operational Data
```

```
↓

Business KPIs

↓

Analytics Engine

↓

Executive Dashboards

↓

Strategic Decisions
```

Business Anomaly Detection

The platform automatically detects unusual business conditions, such as:

- Sudden increase in open tickets
- Drop in AI confidence
- Spike in failed repairs
- Decline in customer satisfaction
- Unusual technician workload
- Abnormal revenue fluctuations

These indicators help management respond proactively before operational issues become critical.

Chapter Summary

The second section of the **Observability & Monitoring Architecture** extends monitoring beyond infrastructure by introducing distributed tracing, AI observability, application performance monitoring, and business intelligence metrics. Together, these capabilities provide complete visibility into the technical and operational health

of the BiteFixes platform, enabling faster diagnostics, continuous optimization, and informed decision-making.

BiteFixes Enterprise Architecture Manual

Volume 16

Observability & Monitoring Architecture (OMA)

16.12 Alerting Architecture

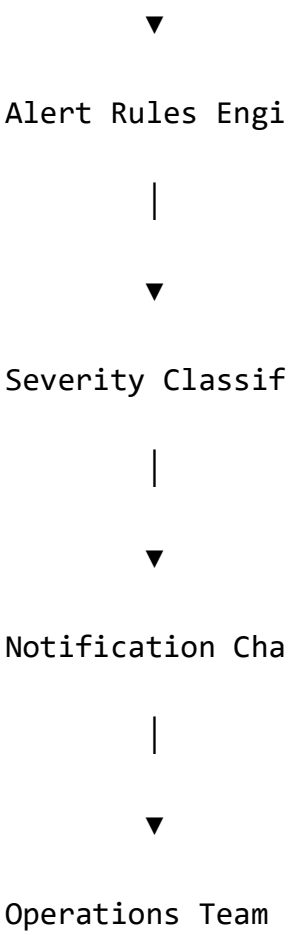
Overview

Monitoring without actionable alerts provides limited operational value. The Alerting Architecture transforms telemetry into timely, meaningful notifications that enable rapid response before issues impact customers.

The objective is to ensure that every critical event generates the appropriate operational response while minimizing alert fatigue.

Alerting Flow

```
Telemetry Sources
        |
        ▼
Monitoring Platform
        |
```



Alert Sources

Alerts may originate from:

- Infrastructure Metrics
- Application Metrics
- API Performance
- Database Performance
- AI Engine
- Security Events
- Business KPIs
- Integration Failures

Severity Levels

Informational

Non-critical operational information.

Examples:

- Scheduled maintenance completed.
- Backup completed successfully.

Warning

Conditions requiring attention but not immediate intervention.

Examples:

- CPU utilization above 75%.
- AI confidence decreasing.
- Response time above normal baseline.

Critical

Events requiring immediate operational response.

Examples:

- Database unavailable.
- API Gateway failure.
- Authentication service offline.
- AI engine unavailable.

Emergency

Platform-wide incidents affecting business continuity.

Examples:

- Complete service outage.
- Data corruption detected.

- Security breach.
- Infrastructure failure.

Alert Routing

Alerts are routed according to severity and responsibility.

```
Application Alerts
        |
        ▼
Application Team

Infrastructure Alerts
        |
        ▼
Infrastructure Team

Security Alerts
        |
        ▼
Security Team

Business Alerts
        |
        ▼
Management

AI Alerts
        |
        ▼
AI Operations Team
```

Alert Suppression

To reduce operational noise, the platform supports:

- Duplicate suppression
- Time-based suppression
- Dependency-aware suppression

- Maintenance windows
- Intelligent grouping

Escalation Policies

Unacknowledged alerts follow predefined escalation paths.

```
Alert Generated

↓

Primary Engineer

↓

Senior Engineer

↓

Operations Manager

↓

Executive Notification
```

16.13 Dashboards

Overview

Dashboards provide real-time visualization of technical, operational, and business health.

Different stakeholders require different views of the platform.

Dashboard Categories

Executive Dashboard

Displays:

- Platform availability
- Revenue trends
- Customer growth
- SLA compliance
- AI automation rate
- Business KPIs

Operations Dashboard

Displays:

- Active incidents
- Infrastructure status
- Service health
- Response times
- Queue status

AI Dashboard

Displays:

- Active conversations
- Intent detection accuracy
- Knowledge retrieval rate
- Memory usage
- AI latency
- Prompt execution statistics

Infrastructure Dashboard

Displays:

- CPU
- Memory
- Storage
- Network
- Database health
- Container status

Security Dashboard

Displays:

- Failed logins
- Authentication events
- Permission violations
- Security alerts
- Audit activity

Dashboard Architecture

```
Telemetry

↓

Metrics

↓

Analytics Engine

↓

Dashboard Services

↓

Web Portal

↓
```

Administrators

Dashboards refresh automatically and support configurable time ranges.

16.14 Capacity Monitoring

Overview

Capacity Monitoring ensures that sufficient computing resources are available to support current workloads and future growth.

Rather than reacting to resource exhaustion, the platform predicts future capacity needs.

Capacity Domains

The following areas are continuously monitored:

- CPU
- Memory
- Storage
- Database
- API Throughput
- AI Processing
- Network
- Queue Depth

Capacity Forecasting

Historical trends are analyzed to estimate future resource requirements.

Historical Metrics

↓

```
Trend Analysis

↓

Forecast Model

↓

Capacity Plan

↓

Infrastructure Expansion
```

Scaling Indicators

Examples include:

- Average CPU utilization above 70%
- Database storage reaching 80%
- Sustained API throughput growth
- AI response latency increasing
- Queue processing delays

These indicators support proactive scaling decisions.

16.15 Incident Detection

Overview

Incident Detection identifies abnormal platform behavior and initiates the incident management process.

The goal is to minimize Mean Time to Detect (MTTD) and Mean Time to Resolve (MTTR).

Detection Sources

Incidents may be detected through:

- Monitoring systems
- Alert rules
- Log analysis
- AI anomaly detection
- User reports
- External monitoring
- Synthetic transactions

Incident Lifecycle

```
Anomaly Detected

↓

Incident Created

↓

Classification

↓

Assignment

↓

Investigation

↓

Resolution

↓

Verification
```

```
↓

Closure

↓

Post-Incident Review
```

Incident Classification

Service Availability

Examples:

- API unavailable
- Database offline
- Infrastructure outage

Performance

Examples:

- Slow API responses
- High database latency
- AI processing delays

Security

Examples:

- Unauthorized access
- Suspicious activity
- Credential compromise

Business

Examples:

- Failed ticket creation
- Payment failures
- Notification failures

Root Cause Analysis

Every major incident includes:

- Timeline
- Root Cause
- Impact Assessment
- Corrective Actions
- Preventive Actions
- Lessons Learned

16.16 Future Evolution

Vision

The Observability Architecture is designed to evolve from reactive monitoring to autonomous operational intelligence.

Future capabilities will include:

- AI-driven anomaly detection
- Predictive incident prevention
- Self-healing infrastructure
- Autonomous performance optimization
- Intelligent alert prioritization
- Automated root cause analysis
- Natural language operational queries
- AI-assisted operational recommendations

Long-Term Architecture

```
Telemetry

↓

AI Analytics

↓

Prediction Engine

↓

Decision Engine

↓

Automated Response

↓

Self-Healing Platform
```

This evolution supports the long-term objective of operating a highly autonomous SaaS platform.

16.17 Volume Summary

The **Observability & Monitoring Architecture (OMA)** establishes a comprehensive framework for monitoring the health, performance, security, and business outcomes of the BiteFixes platform.

The architecture integrates telemetry from infrastructure, applications, APIs, databases, AI services, and business processes into a unified observability ecosystem based on logs, metrics, traces, dashboards, and intelligent alerting.

By combining technical monitoring with AI observability and business intelligence, the platform enables proactive operations, rapid incident response, predictive capacity

planning, and continuous optimization. The long-term vision extends beyond monitoring toward autonomous operational intelligence, where AI assists in detecting anomalies, identifying root causes, and orchestrating self-healing responses.